Deep Learning

MAI 642, DSC 515, ECE 661

Fall 2026

Instructor: Prof. Theocharis Theocharides

Teaching Assistant: Michalis Kontos

Assignment 2

Assignment Date: 30 Sep 2026

Due Date: 14 Oct 2026

Instructions: This assignment includes 2 exercises. You need to submit both your code (if you are using Colab, download the notebook) and a report. The report should include the required plots as well as your answers to the questions. Please submit all of this in one compressed (ZIP) file, named "Assignment2_Yourname" in the corresponding Teams assignment. Ensure to provide clear code, detailed explanations, and visualizations where applicable.

Exercise 1: Training a CNN

In this exercise, you will build a Convolutional Neural Network (CNN) to classify the Fashion MNIST dataset.

Dataset: The Fashion MNIST dataset is a popular benchmark in the field of computer vision and machine learning. It serves as an alternative, more difficult dataset compared to the traditional MNIST. It provides a diverse set of grayscale images that represent various fashion items. Each image is a 28x28-pixel grayscale picture, labeled with a specific class.

Classes: The dataset consists of 10 classes, each corresponding to a distinct fashion item:

1. T-shirt/top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle boot

Dataset Split: The Fashion MNIST dataset is typically divided into training and testing sets:

- Training set: 60,000 labeled images
- Testing set: 10,000 labeled images

Objective:

Your goal is to develop a CNN architecture capable of accurately classifying these fashion items based on the input grayscale images.

First, you will need to load the dataset. You can do that using this code for **TensorFlow**:

```
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical

# Load the Fashion MNIST dataset
(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()

# Normalize the pixel values to be between 0 and 1
train_images = train_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0

# One-hot encode the labels
num_classes = 10
train_labels = to_categorical(train_labels, num_classes)
test_labels = to_categorical(test_labels, num_classes)

# Display the shape of the training and testing data
print("Shape of training images:", train_images.shape)
print("Shape of training labels:", train_labels.shape)
print("Shape of testing images:", test_images.shape)
print("Shape of testing labels:", test_labels.shape)
```

or using **PyTorch** (For PyTorch training with torch.nn.CrossEntropyLoss, you can skip one-hot encoding and keep integer class labels):

```
import torch.nn.functional as F
from torchvision.datasets import FashionMNIST

# Load the Fashion MNIST dataset
train_dataset = FashionMNIST(root="./data", train=True,
download=True)
test_dataset = FashionMNIST(root="./data", train=False,
download=True)

# Normalize the pixel values to be between 0 and 1
train_images = train_dataset.data.to(torch.float32) / 255.0
test_images = test_dataset.data.to(torch.float32) / 255.0
```

```
# One-hot encode the labels
num_classes = 10
train_labels = F.one_hot(train_dataset.targets,
num_classes).float()
test_labels = F.one_hot(test_dataset.targets,
num_classes).float()

# Display the shape of the training and testing data
print("Shape of training images:", train_images.shape)
print("Shape of training labels:", train_labels.shape)
print("Shape of testing images:", test_images.shape)
print("Shape of testing labels:", test_labels.shape)
```

Then, you need to:

i. Design a CNN architecture suitable for image classification. Define the model, including convolutional layers, activation functions, pooling layers, and fully connected layers. You can take inspiration for the model design from some popular CNNs (LeNet, AlexNet etc.)
ii. Train the model on the Fashion MNIST dataset and evaluate its performance using appropriate metrics (e.g., accuracy, loss). Show and discuss your results.

Exercise 2: Residual Networks

In this exercise, you will experiment with Residual Networks (ResNet) by introducing residual connections in your model.

ResNet is one of the most widely adopted CNN architectures, particularly known for effectively addressing the vanishing gradient problem.

i. Briefly explain the concept of residual connections in CNNs and how they address the vanishing gradient problem.
ii. Modify the forward function of the CNN model created in Exercise 1 to incorporate residual connections. Keep in mind that, because of the “pyramid” structure of CNNs where the dimensions are progressively reduced, you need to match the dimensions of the residual connections. You can do that by padding the lower-dimension input (adding 0’s to match the higher dimension).
iii. Implement and train your new model on the Fashion MNIST dataset and compare its performance with the model from Exercise 1.